

Generalized Klondike Solitaire

Project Notes

Status as of July 15, 2026

Contents

1	Executive summary	2
2	Game definition and parameterization	2
3	Player development	3
3.1	Model and training	3
3.2	Current 42 parameters	4
3.3	Information-model qualification	8
4	Exhaustive solvability calculations	8
4.1	Exact-search method	8
4.2	Suit and color automorphisms	9
4.3	Completed results	9
4.4	Smallest unsolvable example	9
5	Stock-order equivalence	10
6	Bounds for standard-game solvability	11
6.1	Deterministic lower bound	11
6.2	Deterministic upper bound	11
6.3	Constructive confidence lower bound	12
7	Validation and reproducibility	13
8	Current conclusions and next steps	13

1 Executive summary

This project developed a parameterized Klondike-style solitaire engine, a trainable move-selection policy, exact small-deck solvers, symmetry reductions, and mathematical bounds for the standard 52-card game.

The main results are:

- The game now supports arbitrary n cards per suit, k suits per color, and t tableau piles, with two colors and $2k$ suits in total.
- The natural default number of tableau piles is $t = \lceil (\sqrt{1 + 8nk} - 1)/2 \rceil$. This gives the usual seven piles when $(n,k)=(13,2)$.
- The player was expanded from 5, then 17, 31, and finally 42 tunable move features. The current greedy policy won 54.1265% of one million untouched confirmatory standard deals, compared with 53.2941% for the frozen stage-4 31-feature policy on the same deals.
- Solvability was classified exactly for every deal with $k=2$ and $n=1,2,3,4$, after mathematically justified symmetry reductions.
- A stock-order theorem justified a further factor of $6! = 720$ reduction for $n=4$, making the exhaustive calculation practical.
- For the standard game, the exact solvability probability remains unknown. We proved a deterministic upper bound of 98.8189535% and, using the frozen 31-feature policy with bounded exact fallback, obtained a 99.9999%-confidence constructive lower bound of 65.9664123%.

The policy win rate and the solvability probability are different quantities. A policy win is one particular algorithm finding a solution; a deal is solvable if any legal sequence of moves wins when the complete deal is known.

2 Game definition and parameterization

The generalized deck has:

- n ranks in each suit;
- k suits of each color;
- two colors, hence $2k$ suits and $2nk$ distinct cards;
- t tableau piles.

The standard game is $(n,k,t)=(13,2,7)$, giving 52 cards, four suits, and seven tableau piles.

For a triangular tableau with piles of sizes 1 through t , the tableau starts with $t(t+1)/2$ cards. We chose the default t to be the smallest integer for which the tableau holds at least half of the deck:

$$\frac{t(t+1)}{2} \geq nk.$$

Solving this inequality gives

$$t = \left\lceil \frac{\sqrt{1 + 8nk} - 1}{2} \right\rceil.$$

This gives $t=2,3,3,4$ for $n=1,2,3,4$ when $k=2$, and $t=7$ for the standard case $n=13$, $k=2$. The value of t can always be overridden.

The rules used throughout the exact calculations and standard-game bounds are:

- draw one card from the stock;
- at most three recycles, hence at most four stock passes;
- movable packed, face-up tableau stacks;
- alternating colors and descending ranks in the tableau;
- only the highest rank may enter an empty tableau pile;
- no moves from a foundation back to the tableau.

Parameter records and evaluation data are stored by the key (n,k,t) , so tuning for one deck shape does not overwrite results for another.

3 Player development

3.1 Model and training

Every legal move is represented by a vector of strategic features. The player assigns the move a linear score equal to the weighted sum of those features. For evaluation it chooses the highest-scoring move. For stochastic training it uses a softmax distribution over legal moves.

The training system supports policy-gradient updates, online stochastic gradient ascent, batched Adam, gradient clipping, and selectively freezing features. This was used during the earlier 5- and 17-feature stages. The final standard-game policy was tuned with common-random coordinate ascent in the fast native evaluator. Using the same random deals for each candidate substantially reduces comparison noise, and coordinate ascent directly targets the discontinuous greedy win count.

The initial 31-feature search tuned the 14 new features first and then refined all 31 weights. Stages 3 and 4 used fresh data and shrinking coordinate steps, eventually producing the 53.2497% stage-4 policy. Stage 5 made a still finer 31-feature continuation on 250,000 training and 500,000 validation deals; its selected round improved validation from 53.1750% to 53.2058%. Round-zero checkpoints ensured that every continuation could retain its starting policy if tuning failed to generalize.

Stage 6 introduced 11 additional tactical features. They distinguish foundation-ready cards from tableau-playable cards, measure the usefulness and foundation distance of newly revealed cards, represent waste timing across limited passes, count opportunities to clear a column, and grade foundation imbalance and safety. The new features were first tuned alone on 200,000 training and 400,000 validation deals, raising validation from 53.3980% to 53.9065%. Stage 7 then refined all 42 weights on another 250,000 training and 500,000 validation deals, raising its fresh validation result from 53.8466% to 54.1322%.

On a one-million-deal selection seed, stage 4, stage 5, stage 6, and stage 7 won 53.3041%, 53.3898%, 53.9036%, and 54.1525%, respectively. Stage 7 was then frozen before the final confirmatory seed was evaluated:

Policy	Test role	Wins	Deals	Win rate	Average moves
Frozen stage-4 31-feature policy	Confirmatory baseline	532,941	1,000,000	53.2941%	117.711030
Current stage-7 42-feature policy	Confirmatory candidate	541,265	1,000,000	54.1265%	120.947843

The current policy added 8,324 wins per million deals over stage 4, a gain of 0.8324 percentage points or 1.5619% relative. Average game length increased by 2.7498%, a modest computational tradeoff for the stronger policy. In the earlier held-out comparison, expanding the 17-feature policy to the first 31-feature policy added 46,548 wins and 4.6548 percentage points.

The smaller-deck work provided useful development checks. The original five-feature policy won 95.64% of 5,000 $n=3$ evaluation deals. A tuned 17-feature $n=4$ policy won about 95.23% of one million sampled deals and was used as a constructive front end to the exact solver. These percentages are high partly because almost all deals at those deck sizes are solvable.

3.2 Current 42 parameters

The table below gives the selected weights for $(n,k,t)=(13,2,7)$, ordered by empirical contribution rather than coefficient magnitude. The stock is the face-down packet from which cards are drawn; each draw places one card face up on the waste, and only the exposed top waste card can be played. The tableau consists of the seven working columns, while each suit’s foundation is built upward from rank 1. A card is “foundation-ready” when it is the next required rank for its suit.

Every feature is computed for one candidate move. Binary features are either 1 when their stated condition holds or 0 otherwise; count and distance features are 0 when they do not apply. The examples use standard-game ranks and colors, and the reported example number is the feature value before multiplication by its weight. The move score is the sum of each feature value multiplied by its weight, so a positive weight favors larger feature values and a negative weight discourages them, all else being equal.

Each feature was ablated by setting only its weight to zero and replaying the same 500,000 fresh deals (seed 940001); the full model won 271,019 games, or 54.2038%. “Wins lost” is the full-model win count minus the ablated win count, so larger positive values indicate greater contribution in this fitted model, while a negative value means the zero-weight variant won more on this sample. Ablation effects are conditional on the other 41 weights, are not additive, and should not be interpreted as independent causal effects. “Model #” preserves the feature’s code and parameter-file index. Full results are stored in `brute_force/results/k2_n13_t7.feature-ablation.json`.

Rank	Model #	Parameter	Description	Weight	Wins lost
1	5	<code>stock_action</code>	Binary: the candidate either draws the next stock card or recycles the waste into a new stock. It is 0 for every move that plays a card or stack. Example: drawing the next stock card gives 1, while moving a black 7 from the waste onto a red 8 gives 0.	-0.960810	99,961
2	9	<code>waste_to_tableau</code>	Binary: the candidate moves the exposed waste card onto a tableau pile, rather than drawing past it or moving it directly to its foundation. Example: moving an exposed black 7 from the waste onto a red 8 gives 1.	0.937500	85,891
3	2	<code>reveal_hidden</code>	Binary: removing a card or stack from a tableau pile uncovers and turns face up the next card in that source pile. Example: moving a face-up black 7 away when the card directly beneath it is face down gives 1.	4.875000	41,896

Continued on next page

Rank	Model #	Parameter	Description	Weight	Wins lost
4	4	tableau_build	Number of cards the candidate places onto a tableau pile. A waste-to-tableau move has value 1, while a tableau stack transfer has the full length of the moved stack. Example: moving a three-card black queen, red jack, black 10 stack onto a red king gives 3.	-1.131250	27,189
5	28	next_reveal_moves	After applying the candidate, the number of legal follow-up moves that would uncover a face-down tableau card. This measures how many immediate routes to further hidden information the move creates. Example: if the resulting position has two legal moves that each uncover a hidden card, the value is 2.	2.312500	15,944
6	23	productive_stack_length	Length of a tableau stack transfer when that transfer either reveals a hidden card or empties its source column. It is 0 for all other moves, including tableau transfers that accomplish neither objective. Example: moving a three-card stack and thereby exposing a face-down card gives 3; moving the same stack without revealing or emptying gives 0.	0.693750	15,733
7	39	waste_play_recycle_pressure	For a move from the waste to a foundation or tableau pile, the number of recycles already consumed: 0 on the initial pass, 1 after the first recycle, and so on. It is 0 for non-waste moves. Example: playing a waste card during the initial pass gives 0, while playing it after two recycles gives 2.	0.296875	13,101
8	27	next_foundation_moves	After applying the candidate, the number of legal follow-up moves from the waste or tableau to any foundation. It measures the immediate foundation opportunities left or created by the move. Example: if an ace is exposed on the waste and another ace is exposed on the tableau afterward, the value is 2.	0.903125	7,784
9	25	waste_unlocks_playable	Binary: playing the current waste card exposes the card immediately beneath it, and that newly exposed card can immediately move either to its foundation or to at least one tableau pile. Example: playing the top waste card and uncovering an ace that can immediately move to its foundation gives 1.	0.812500	6,886
10	31	draw_stock_remaining	For a draw, the number of cards in the stock immediately before the draw, including the card about to be drawn. It is 0 for other moves and distinguishes early-pass draws from draws near the end of the stock. Example: drawing when 12 cards remain in the stock gives 12.	0.037500	5,738
11	7	reveal_depth	When the candidate reveals a hidden tableau card, the total number of face-down cards in that source pile before the move; otherwise 0. Larger values identify progress in more deeply blocked columns. Example: removing the face-up stack from a column containing three face-down cards gives 3 when the move turns the top hidden card face up.	0.968750	3,202
12	24	draw_playable	Binary: after the draw, the newly exposed waste card can immediately move to its foundation or can legally build on at least one tableau pile. Example: drawing a black 7 when a red 8 is exposed on the tableau gives 1.	2.000000	2,855
13	10	recycle	Binary: the candidate consumes one of the limited recycles by turning the waste over to form a new stock. Draws and card-playing moves have value 0. Example: turning an exhausted waste pile over for another pass gives 1; drawing an ordinary card gives 0.	-1.812500	2,315

Continued on next page

Rank	Model #	Parameter	Description	Weight	Wins lost
14	1	safe_foundation	Binary for a foundation move: the card is rank 1, or every opposite-color foundation has reached at least one rank below it. This is the usual safety test for avoiding premature removal of a card that may still be needed as tableau support. Example: moving the 6 of clubs to its foundation is safe when both red foundations have reached at least rank 5, so the value is 1.	9.937500	2,090
15	29	foundation_support_demand	For a card being sent to its foundation, the number of visible opposite-color cards exactly one rank lower, counted across all face-up tableau cards and the exposed waste card. Those cards could still need the departing card as a tableau parent. Example: when a black 8 moves to its foundation while both red 7s are visible, the value is 2.	-0.781250	1,921
16	11	destination_hidden	For a move onto the tableau, the number of face-down cards buried in the destination column before the move. It is 0 for moves whose destination is a foundation or the stock. Example: placing a card onto a column containing four face-down cards beneath its visible run gives 4.	-0.061250	1,377
17	15	source_hidden	For a move originating in the tableau, the number of face-down cards in its source column before the move. It is 0 for draws, recycles, and moves from the waste. Example: moving a stack from a column with five face-down cards gives 5.	0.375000	1,157
18	17	first_empty_with_king	Binary: the candidate creates the first empty tableau column in a position that previously had none, and the resulting position contains a legal move of a highest-rank card or stack into that empty column. Example: clearing a column when all seven were occupied and an exposed king can immediately fill it gives 1.	1.406250	895
19	30	recycle_pressure	For a recycle, the ordinal number of the recycle about to be consumed: 1 for the first recycle, 2 for the second, and 3 for the third under standard rules. It is 0 for non-recycle moves. Example: choosing the second recycle gives 2.	-0.406250	793
20	38	draw_buries_playable_waste	Binary: the candidate draws even though the currently exposed waste card already has a legal move to a foundation or tableau pile, so the draw covers a card that could have been played now. Example: drawing while an exposed black 7 could be placed on a red 8 gives 1.	0.125000	556
21	16	creates_first_empty	Binary: every tableau column is occupied before the candidate, and the candidate moves every card out of one source column, thereby creating the position's first empty column. Example: when all seven columns are occupied, moving the entire contents of one column elsewhere gives 1.	0.875000	543
22	12	destination_run	For a move onto the tableau, the number of face-up cards already in the destination column before the move. This is the length of the existing visible run that the new card or stack will cover. Example: building onto a column whose visible run contains three cards gives 3.	-0.037500	529
23	42	unsafe_foundation_distance	For a foundation move of rank r , the number of ranks by which it exceeds the opposite-color safety frontier: $\max(0, r - 1 - m)$, where m is the least advanced opposite-color foundation. A fully safe move has value 0. Example: moving a black 8 when the lower red foundation is at rank 5 gives $8 - 1 - 5 = 2$.	-0.093750	477
24	20	tableau_to_foundation	Binary: the candidate moves the exposed top card of a tableau column directly to its suit foundation. Foundation moves from the waste and all non-foundation moves have value 0. Example: moving the 5 of hearts from a tableau column to its foundation gives 1, while moving it there from the waste gives 0.	-0.768750	460

Continued on next page

Rank	Model #	Parameter	Description	Weight	Wins lost
25	8	empty_source	Binary: a tableau-to-tableau or tableau-to-foundation move removes every remaining card from its source column, leaving that tableau column empty. Example: moving the only two-card stack out of a column gives 1.	-0.250000	366
26	13	blocks_foundation	Binary: a move onto the tableau covers a destination card that, before the move, was exposed and legally able to move to its foundation. It identifies the immediate loss of that foundation option. Example: placing a black 5 on an exposed red 6 that could already move to its foundation gives 1.	-0.718750	296
27	34	draw_foundation_ready	Binary: the card exposed on the waste by the candidate draw is already the next required card for its suit and can therefore move immediately to its foundation. Example: if the hearts foundation is at rank 4, drawing the 5 of hearts gives 1.	-1.500000	256
28	36	waste_unlocks_foundation_ready	Binary: after playing the current waste card, the card newly exposed beneath it is exactly the next required card for its suit and can immediately move to its foundation. Example: playing the top waste card to uncover the 5 of hearts when that foundation is at rank 4 gives 1.	-1.843750	237
29	32	revealed_card_tableau_moves	When the candidate uncovers a hidden tableau card, the number of tableau columns onto which that newly revealed card could legally move in the resulting position; otherwise 0. Example: revealing a black 7 when two red 8s are exposed on different columns gives 2.	0.812500	96
30	19	revealed_card_foundation_ready	Binary: the tableau card turned face up by the candidate is the next required card of its suit and can move immediately to its foundation. Example: revealing the 5 of hearts when the hearts foundation is at rank 4 gives 1.	1.250000	93
31	40	next_empty_source_moves	After applying the candidate, the number of legal follow-up moves originating in the tableau that would remove every card from their source column, whether the cards move to another column or to a foundation. Example: if the resulting position has three different legal moves that each clear a source column, the value is 3.	-0.125000	84
32	33	revealed_card_foundation_distance	For a newly revealed tableau card, the number of lower cards of its own suit that must still reach the foundation before it becomes foundation-ready. Formally this is its rank minus the next required foundation rank, with a minimum of 0. Example: revealing the 8 of hearts when that foundation is at rank 5 gives $8 - 6 = 2$, because the 6 and 7 must move first.	0.031250	58
33	18	revealed_card_low_rank	A reverse-rank value for a newly revealed tableau card: $n + 1 - \text{rank}$, so an ace has value n and a highest-rank card has value 1. It is 0 when the candidate reveals no hidden card. Example: in the standard game, revealing an ace gives 13, while revealing a king gives 1.	-0.050000	54
34	37	waste_unlocks_tableau_moves	After playing the current waste card, the number of tableau columns onto which the card newly exposed beneath it could legally move. It is 0 if no waste card is uncovered or none of its tableau moves is legal. Example: uncovering a black 7 beneath the played waste card when two red 8s are exposed gives 2.	0.125000	46
35	6	foundation_move	Binary: the candidate moves a card to a foundation, whether the card comes from the tableau or the waste. Every draw, recycle, and tableau-building move has value 0. Example: moving an ace from the waste to its foundation gives 1.	0.031250	1
36	14	foundation_rank	Numerical rank of the card moved to a foundation, with ace represented by 1 and the highest rank by n . It is 0 for every move that does not place a card on a foundation. Example: moving a queen to its foundation gives 12 in the standard game.	-0.006250	-5

Continued on next page

Rank	Model #	Parameter	Description	Weight	Wins lost
37	21	<code>tableau_to_tableau</code>	Binary: the candidate transfers one card or a legal descending alternating-color stack from one tableau column to another. All waste plays, foundation moves, draws, and recycles have value 0. Example: moving a three-card stack between columns gives 1, not 3.	-0.015625	-9
38	35	<code>draw_tableau_moves</code>	After a draw, the number of tableau columns onto which the newly exposed waste card could legally be played. It is 0 for non-draw moves or when the drawn card has no tableau destination. Example: drawing a black 7 when two red 8s are exposed gives 2.	0.125000	-70
39	41	<code>foundation_lag</code>	For a foundation move, the difference between the height of the most advanced foundation and the pre-move height of this card's own foundation. A value of 0 means its foundation is tied for the lead; larger values mean it is catching up from behind. Example: moving a 7 when its foundation is at rank 6 and the leading foundation is at rank 9 gives $9 - 6 = 3$.	-0.037500	-78
40	26	<code>empty_king_queen_access</code>	When the candidate fills an empty column with a highest-rank card or stack, the number of legal follow-up moves that can place an accessible card of the next lower rank onto that same column. It is 0 for moves that do not fill an empty column this way. Example: after moving a black king into an empty column, one accessible red queen on the waste and another on the tableau give 2.	0.656250	-163
41	3	<code>empty_king</code>	Binary: the candidate fills an empty tableau column with a highest-rank card, or with a movable stack whose bottom card has the highest rank. Example: moving a king-led stack into an empty column gives 1.	0.312500	-422
42	22	<code>non_reveal_tableau_move</code>	Binary: the candidate transfers a card or stack between tableau columns but neither uncovers a face-down card nor empties its source column. It marks a rearrangement with no immediate hidden-card or empty-column gain. Example: moving a black 7-red 6 stack onto a red 8 while its source retains an exposed card gives 1.	-0.968750	-488

3.3 Information-model qualification

The current player is best regarded as a perfect-information heuristic solver, not yet as a model of human play. Candidate moves are evaluated by simulating their immediate successor states. Consequently, features such as `draw_playable`, `revealed_card_low_rank`, and `revealed_card_foundation_ready` can use the identity of a card that is still face down before the move. Some one-step opportunity counts inherit the same information.

This makes the 54.1265% result useful as a constructive solver benchmark, but it is not directly comparable with the reported human win rate of roughly 18%. A human-information policy would need to mask hidden identities, replace these features with beliefs or observable proxies, and then be retuned.

4 Exhaustive solvability calculations

4.1 Exact-search method

The exact solver explores the complete reachable state graph of a deal. If it reaches a win, the deal is solvable; if the complete reachable graph is exhausted without a win, the deal is unsolvable. Previously classified states are cached.

The learned policy is used only as a constructive shortcut. A model win is a complete legal path and therefore proves solvability. Every model failure is sent to exact search. The policy can improve speed but cannot create a false positive or change an exact classification.

The native solver also introduced several exact optimizations:

- force foundation moves that are provably safe;
- treat tableau-pile order canonically in the state cache;
- remove equivalent moves into multiple empty piles;
- checkpoint large sweeps so they can be resumed.

4.2 Suit and color automorphisms

The rules are unchanged if suits are permuted within a color or if the two colors are exchanged. For general k , this symmetry group has size

$$2(k!)^2.$$

For $k=2$, every orbit has eight deals. We classify only the lexicographically smallest deal in each orbit, reducing every exhaustive calculation by a factor of eight without changing solvability.

The outcomes are stored as one bit per reduced deal, then compressed. Metadata records the deterministic deal order and both compressed and uncompressed checksums. This gives a compact exact data set that can later be replayed against any player.

4.3 Completed results

n	k	t	Raw deals	Reduced classes searched	Solvable raw deals	Unsolvable raw deals	Solvability rate
1	2	2	24	3	24	0	100.000000%
2	2	3	40,320	5,040	40,032	288	99.285714%
3	2	3	479,001,600	59,875,200	475,004,160	3,997,440	99.165464%
4	2	4	20,922,789,888,000	3,632,428,800	20,617,111,307,520	305,678,580,480	98.539016%

For $n=4$, the reduced classes additionally identify all $6!$ orders of the six stock cards. That reduction is justified by the stock-order theorem below; it is not merely a heuristic sampling assumption.

The complete $n=4$ sweep took about 93 minutes on eight threads and expanded 72,264,410,042 positions. Its one-bit data set occupies 454,053,600 bytes before compression and 8,517,753 bytes after compression.

4.4 Smallest unsolvable example

Every $n=1$ deal is solvable, so $n=2$ is the smallest deck size with losses. One canonical unsolvable deal uses the ranks Ace and 2 in each standard suit. Reading each tableau pile from bottom to top, its initial layout is:

Pile	Cards
1	2 of Clubs, face up
2	Ace of Clubs, face down; 2 of Diamonds, face up
3	Ace of Diamonds, face down; Ace of Spades, face down; 2 of Spades, face up
Stock draw order	Ace of Hearts, then 2 of Hearts

After the Heart cards are played, the visible rank-2 tableau cards form a dependency deadlock over the hidden Aces, and no empty pile is available. Exact state-graph search confirms that no legal winning path exists. There are 36 unsolvable canonical $n=2$ deals, representing 288 raw deals.

5 Stock-order equivalence

For a full triangular tableau, let

$$m = \max \left(0, 2nk - \frac{t(t+1)}{2} \right)$$

be the number of stock cards, and let R be the number of permitted recycles. An initial pass plus R recycles gives $R+1$ total passes.

Suppose an unordered-reserve version of a deal has a winning solution. That solution removes the m reserve cards in some target order. For any actual stock ordering, draw until the next target card is exposed on the waste, play it, reproduce the intervening tableau and foundation moves from the reserve solution, and recycle when necessary. Cards waiting in the waste do not affect tableau or foundation legality. By induction, each played target card reaches the same location it occupied in the reserve solution, so the complete reserve win can be replayed if enough passes are available. The converse is immediate: every ordered-stock win is also an unordered-reserve win.

Let $P(m)$ be the maximum number of passes needed, over all relative stock orders, to reproduce an arbitrary prescribed removal order. Stock order and an unordered reserve are therefore equivalent whenever

$$R + 1 \geq P(m).$$

For every m , we proved the general bound $P(m) \leq m$: each complete pass can remove at least the next required card. Thus $m-1$ recycles are always sufficient for stock-order equivalence under draw-one rules.

Finite enumeration gives the sharper small-stock values:

Stock cards m	1	2	3	4	5	6	7	8	9	10
Required passes $P(m)$	1	1	2	2	3	4	5	5	6	7

For six stock cards, all $6! = 720$ relative orders were checked. The pass distribution was 132 orders needing one pass, 424 needing two, 160 needing three, and four needing four. Since this project allows four passes, all six stock cards may be treated as an unordered reserve. This applies to both $n=3$ and $n=4$; for $n=4$ it supplies the decisive extra factor of 720.

The guarantee ends at six stock cards under the four-pass rules. A seven-card relative order exists that needs five passes. This proves that the reserve replay argument cannot collapse stock order once $m \geq 7$; it does not by itself prove that every larger game is sensitive to stock order.

The standard game has 24 stock cards. The simple theorem would guarantee order equivalence with 23 recycles, but the actual game allows only three. Therefore the standard stock cannot be treated as unordered by this theorem.

6 Bounds for standard-game solvability

Let p be the probability that a uniformly random standard deal is solvable under the rules stated above. Solvability assumes the full deal is known and asks whether at least one legal winning sequence exists.

6.1 Deterministic lower bound

Fix a total order of the 52 cards that respects Ace-through-King order within each suit. Read each initial tableau pile from its exposed card downward, and read the stock in draw order. These form eight availability chains of lengths 1, 2, 3, 4, 5, 6, 7, and 24.

If every chain agrees with the fixed total order, the cards can be moved directly to the foundations in that order. The fraction of deals satisfying these relative-order constraints is

$$p \geq \frac{1}{1!2!3!4!5!6!7!24!} \approx 1.285161075 \times 10^{-35}.$$

This bound is extremely small, but it is unconditional and every counted deal is certifiably solvable.

6.2 Deterministic upper bound

A tableau card of rank 2 through Queen is permanently blocked if the cards beneath it in the same initial pile include:

1. a lower-ranked card of the same suit; and
2. both opposite-color cards one rank higher.

The same-suit card prevents the target from reaching its foundation, while both possible tableau supports are inaccessible beneath it. The target is not a King and therefore cannot enter an empty pile. Any deal containing this configuration is unsolvable.

There are 440 eligible target-card and tableau-position events. Exact integer counting gives the sum of individual event probabilities and all pairwise intersections. Applying the second-order Bonferroni inequality proves

$$\Pr(\text{unsolvable}) \geq \frac{516163096789}{43703874396000} = 0.011810465408903011.$$

Therefore

$$p \leq \frac{43187711299211}{43703874396000} = 0.988189534591097.$$

6.3 Constructive confidence lower bound

A fresh, fixed confirmatory solver used the frozen 31-feature stage-4 policy, followed when necessary by exact search capped at 100,000 expanded nodes. The policy, search budget, confidence level, and independent shuffle seed were fixed before inspecting the outcomes. The run used eight threads, took 432.871 seconds, and expanded 679,980,828 exact-search positions. Its complete outcome accounting was:

Outcome	Deals
Won by the 31-feature policy alone	10,631
Won by exact fallback	2,913
Proved unsolvable by exact fallback	110
Exact budget exhausted; unresolved	6,346
Total	20,000

Thus exact search was invoked on 9,369 policy failures, and the combined solver found 13,544 complete legal wins. The 6,346 budget-exhausted deals were left unresolved, not counted as unsolvable. Each of the 13,544 successes includes a legal winning path, so the fixed solver’s true success probability q cannot exceed the game’s solvability probability p .

The observed constructive success rate was $13,544/20,000 = 67.72\%$. Inverting a one-sided Bernoulli KL-Chernoff bound at significance level 10^{-6} gives a lower confidence endpoint of 0.6596641225626343 for q . Since $q \leq p$, the same endpoint is a valid lower confidence bound for solvability:

$$p \geq 0.6596641225626343$$

with 99.9999% confidence under the interpretation of the pseudorandom shuffles as independent uniform deals. Combining this with the deterministic upper bound gives

$$65.9664123\% \leq p \leq 98.8189535\%$$

with 99.9999% confidence for the lower endpoint. The upper endpoint is fully deterministic. Without any sampling assumption, the rigorous interval remains the much wider interval from approximately 1.285×10^{-35} to 98.8189535%.

The 67.72% observed rate is a solver success rate, not an estimate that only 67.72% of deals are solvable. Budget-exhausted deals can contain further wins. The exact value of p remains unknown. Blake and Gent reported 81.945% +/- 0.084% for a related thoughtful Klondike variant, suggesting that a value in the low 80s is plausible, but their rules differ and their result is context rather than a bound for this game.

The archived 17-feature run had found 13,012 wins and produced the lower endpoint 63.2745638%. The new independent run raises the confidence lower endpoint by about 2.69185 percentage points. A still tighter endpoint would require another preregistered independent sample, a larger sample size, or a stronger solver frozen before its confirmatory run.

The current 42-feature stage-7 player was developed after this constructive-bound protocol and sample were completed. Its higher player win rate does not retroactively alter the stated confidence bound. A new bound would require freezing the 42-feature solver and evaluating it with an independent constructive sample and a newly specified fallback protocol.

7 Validation and reproducibility

The following checks were completed:

- The current Python and native 42-feature players produced identical win/loss bitsets on all 5,040 canonical $n=2$ deals.
- The native exact move engine was also checked deal for deal against the Python exact solver on the complete canonical $n=2$ set and on mixed $n=3$ samples containing known losses.
- The project test suite contains 34 passing tests, including checks for the new tactical features, parameter persistence, symmetry counts, stored exact data, stock-order bounds, and standard probability calculations.
- The native solver builds without compiler warnings.
- Exhaustive bitsets include deterministic enumeration metadata and SHA-256 checksums, allowing later player evaluations to use exactly the same deals.
- Each continuation used fresh training and validation seeds. Stage 7 was selected on seed 920001 and then frozen before its final paired million-deal confirmation on seed 930001.
- The constructive solvability sample used another fresh seed after the 31-feature policy and 100,000-node fallback protocol had been frozen.

8 Current conclusions and next steps

1. Small generalized games are overwhelmingly but not universally solvable. Exact rates fall from 100% at $n=1$ to 98.539016% at $n=4$.
2. Rule-preserving deck automorphisms and the proved stock-order quotient make exhaustive work several orders of magnitude smaller without changing the question being answered.
3. The 42-feature linear policy is substantially stronger than the frozen 31-feature version on standard deals, but it remains far below a strong perfect-information solver and uses information unavailable to a human.
4. The standard solvability probability is certainly below 98.819%, and is at least 65.966% with very high confidence, but the present interval is still wide.
5. The most useful next empirical step is a larger preregistered sample or a stronger frozen constructive solver. The most useful player-development step is to separate perfect-information solver features from an imperfect-information policy intended for human comparison.
6. Exhaustive $n=5$ classification is much harder: the default game has ten stock cards, for which four passes do not support the stock-order quotient, and the raw deal space grows dramatically.

The related thoughtful-Klondike comparison is from Blake and Gent, "The Winnability of Klondike Solitaire and Many Other Patience Games," arXiv:1906.12314.